NEXUS

MANUAL TÉCNICO Y PREPARACIÓN PARA EL EXAMEN H3

Programación Orientada a Objetos · C#

Repositorio: https://github.com/dragon-51/nexus

Guía de estudio basada en el código publicado, con ejemplos complementarios claramente identificados.

Documento académico preparado el 12 de octubre de 2026. Verificar nuevamente el código si el repositorio cambia.

Introducción

NEXUS es una aplicación de consola escrita en C# que simula la exploración de realidades, la administración de un cadete, un mapa matricial, un inventario y encuentros con enemigos. El punto de entrada crea un objeto `SistemaNexus` y llama a `Ejecutar()`. El sistema solicita los datos iniciales, selecciona un mapa, comprueba los valores de energía y estabilidad, intenta establecer una conexión y después inicia el bucle principal.

El archivo revisado concentra las clases principales en `Program.cs`. Por ello, cuando se habla de la estructura del proyecto, la separación descrita aquí es principalmente lógica (responsabilidades por clase), no necesariamente una separación en archivos distintos.

I. Arquitectura y programación orientada a objetos

1. Propósito y funcionamiento general

El propósito de NEXUS es ejecutar una simulación interactiva desde la consola. El usuario crea un cadete, selecciona una realidad y puede desplazarse por el mapa, revisar y utilizar objetos, recuperar energía, combatir enemigos y completar la misión al localizar el núcleo energético.

El flujo general implementado es:

1. El punto de entrada crea `SistemaNexus`.

2. `SistemaNexus.Ejecutar()` solicita identificación, edad, realidad, energía y estabilidad.

3. Se instancia `Cadete`; el constructor inicializa la salud en 100 y el inventario inicial.

4. Si la energía es menor que 50 o la estabilidad es menor que 20, el acceso se rechaza.

5. Si supera esa validación, se procesa el intento de conexión y comienza `BucleJuegoPrincipal()`.

6. El menú permite moverse, abrir inventario, diagnosticar, recuperar energía o intentar extracción.

7. Durante la exploración se validan límites y obstáculos; la casilla puede activar eventos como combate, objetos, pérdida de energía o la misión.

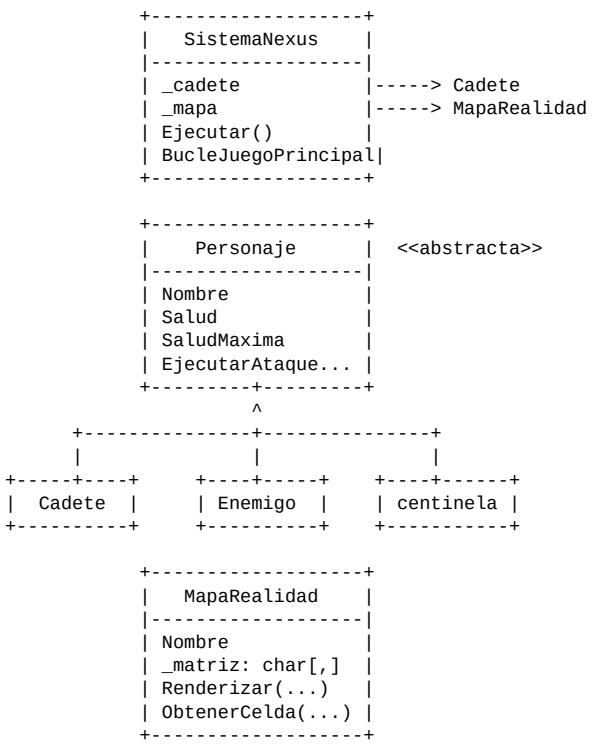
La conexión de red intenta contactar con `192.168.40.11:5000`. Si falla, el programa captura la excepción y comunica que continuará en modo de simulación local.

2. Estructura y responsabilidades de las clases

Todas las clases principales identificadas se encuentran en `Program.cs`.

Clase / elemento	Responsabilidad observada en el código
`Program`	Punto de entrada; crea `SistemaNexus` e inicia su ejecución.
`SistemaNexus`	Coordina el flujo, mantiene referencias al cadete y al mapa, procesa menús, exploración, eventos, combate y estado de misión.
`Personaje`	Clase abstracta base; define `Nombre`, `Salud`, `SaludMaxima`, un constructor y el método abstracto `EjecutarAtaqueEnemigo()`.
`Cadete`	Especializa `Personaje`; administra edad, energía, estabilidad, experiencia, posición e inventario. También muestra el inventario.
`Enemigo`	Especializa `Personaje`; implementa ataques aleatorios y devuelve una cantidad de daño.
`centinela`	Especializa `Personaje`; tiene su propio nombre y salud iniciales y otro conjunto de ataques aleatorios. El nombre de la clase está escrito en minúscula en el código.
`MapaRealidad`	Guarda nombre y matriz del mapa; expone dimensiones, lectura y modificación de celdas y representación visual.
`InterferenciaIrisHandler`	Delegado que define la forma de la notificación de anomalía: mensaje y valor de entropía.

3. Diagrama de clases y relaciones



La flecha desde `Cadete`, `Enemigo` y `centinela` hacia `Personaje` representa herencia. `SistemaNexus` usa y mantiene referencias a `Cadete` y `MapaRealidad`; esto es una relación de asociación/uso, no herencia. El diagrama es conceptual y no representa todos los métodos auxiliares.

4. Programación estructurada frente a POO

La programación estructurada organiza el flujo alrededor de secuencias, condiciones, bucles y procedimientos. La POO agrupa estado y comportamiento en objetos que colaboran entre sí.

En NEXUS aparecen ambas ideas: `SistemaNexus` utiliza `if`, `switch`, `while` y métodos para organizar el flujo, mientras que las entidades `Cadete`, `Personaje`, `Enemigo` y `MapaRealidad` modelan datos y comportamientos. POO no elimina las condiciones ni los bucles; los utiliza dentro de una arquitectura basada en clases.

II. Fundamentos de clases y objetos

1. Clases, objetos e instanciación

Una clase define datos y comportamientos; un objeto es una instancia concreta de esa clase. En `SistemaNexus.Ejecutar()` se crea el cadete con una instrucción equivalente a:

```
_cadete = new Cadete(nombre, edad, energia, estabilidad);
```

En combate, el sistema también puede crear un objeto de `centinela` o `Enemigo` y pasarlo al método `IniciarFaseCombatePokemon(Personaje enemigo)`. Eso permite que el combate reciba cualquiera de las clases derivadas mediante una referencia de tipo base.

2. Campos, propiedades, métodos y constructores

• **Campo:** variable declarada dentro de una clase, como `private char[,] _matriz` en `MapaRealidad`.

• **Propiedad:** miembro que expone lectura y/o escritura controlada, como `public int Edad { get; private set; }` en `Cadete`.

• **Método:** operación que ejecuta una acción, como `MostrarInventario()`, `Renderizar()` o `Ejecutar()`.

• **Constructor:** inicializa el objeto. `Cadete` recibe nombre, edad, energía y estabilidad, y llama a `base(nombre, 100)`.

• **Sobrecarga:** varios constructores o métodos con el mismo nombre, pero con listas de parámetros distintas. En el `Program.cs` revisado no se identifica una sobrecarga de constructores para estas clases; se explica aquí como concepto teórico.

Ejemplo real simplificado del constructor de `Cadete`:

```
public Cadete(string nombre, int edad, int energia, int estabilidad)
    : base(nombre, 100)
{
    Edad = Math.Clamp(edad, 16, 100);
    Energia = Math.Clamp(energia, 0, 100);
    Estabilidad = Math.Clamp(estabilidad, 0, 100);
    Experiencia = 0;
    Inventario = new List<string> { "Célula de energía", "Escudo" };
}
```

`Math.Clamp` limita los valores al rango indicado. La salud y salud máxima se inicializan en la clase base mediante `base(nombre, 100)`.

3. Encapsulamiento y modificadores de acceso

El encapsulamiento controla cómo se lee o modifica el estado de un objeto. En el proyecto, `Edad` e `Inventario` tienen `private set`: otros objetos pueden leer la propiedad, pero no asignarle un nuevo valor directamente mediante el setter.

Modificador	Significado general
`public`	Accesible desde código que puede acceder al tipo.
`private`	Accesible solo dentro del tipo que lo declara.
`protected`	Accesible dentro del tipo y sus clases derivadas.
`internal`	Accesible dentro del mismo ensamblado.

En `MapaRealidad`, `_matriz` es privada y las operaciones de consulta/modificación se exponen mediante métodos. Sin embargo, la matriz se recibe en el constructor y se guarda como referencia; el encapsulamiento del campo no implica por sí solo una copia defensiva de la matriz.

4. Uso de `this` y `base`

`base` sí aparece en NEXUS. Por ejemplo, `Cadete` llama al constructor de `Personaje` con `base(nombre, 100)`. Los constructores de `Enemigo` y `centinela` también llaman al constructor base para establecer nombre y salud iniciales.

`this` se refiere al objeto actual. No es necesario forzar su uso en la explicación si no aparece en el fragmento que se está analizando. Ejemplo teórico complementario:

```
public class Ejemplo
{
    private string nombre;
    public Ejemplo(string nombre)
    {
        this.nombre = nombre; // this.nombre es el campo; nombre es el parámetro
    }
}
```

III. Herencia, polimorfismo e interfaces

1. Clases base y derivadas

`Personaje` es una clase abstracta. Define los datos comunes `Nombre`, `Salud` y `SaludMaxima`, y declara el método abstracto `EjecutarAtaqueEnemigo()`. Las clases `Cadete`, `Enemigo` y `centinela` derivan de ella y aportan su implementación del método.

La herencia evita repetir los datos comunes de un personaje y permite que cada clase especializada implemente un comportamiento distinto.

2. `abstract`, `virtual` y `override`

• `abstract` indica que la clase o el miembro no proporciona una implementación completa que pueda usarse directamente; una clase derivada concreta debe implementar el método abstracto.

• `virtual` permite que un método de la clase base tenga una implementación que una clase derivada puede sobrescribir.

• `override` declara la implementación que reemplaza el miembro virtual o abstracto heredado.

En el código real, `Personaje.EjecutarAtaqueEnemigo()` es `abstract` y las clases derivadas usan `override`. No es necesario que el método base sea `virtual` si ya es `abstract`.

3. Polimorfismo y resolución de llamadas

El método `IniciarFaseCombatePokemon(Personaje enemigo)` recibe un `Personaje`, pero el objeto real puede ser un `Enemigo` o un `centinela`. El código crea el rival de forma aleatoria y lo entrega al mismo método:

```
Personaje rival;
if (_random.Next(1, 3) == 1)
    rival = new centinela();
else
    rival = new Enemigo();

IniciarFaseCombatePokemon(rival);
```

Cuando se invoca `enemigo.EjecutarAtaqueEnemigo()`, C# selecciona la implementación sobrescrita correspondiente al tipo real del objeto. Ese comportamiento es polimorfismo en tiempo de ejecución.

4. Interfaces y contratos — teoría complementaria

Una interfaz no está integrada en la versión revisada del `Program.cs`. Se explica aquí solo como teoría para estudiar la consigna.

Una interfaz declara un contrato que las clases implementadoras deben cumplir. Por ejemplo:

```
public interface IRecuperable
{
    void Recuperar(int cantidad);
}

public class Bateria : IRecuperable
{
    public int Carga { get; private set; }
    public void Recuperar(int cantidad)
    {
        if (cantidad > 0)
            Carga = Math.Min(100, Carga + cantidad);
    }
}
```

Este es un ejemplo complementario, no código existente de NEXUS. `IRecuperable` establece el contrato `Recuperar`; `Bateria` lo implementa.

5. Herencia frente a composición e interfaces

• **Herencia («es un»):** `Enemigo` es un `Personaje`; hereda los miembros comunes y especializa el ataque.

• **Composición («tiene un»):** una clase contiene o administra otro objeto como parte de su funcionamiento. En NEXUS, `SistemaNexus` mantiene referencias a un `Cadete` y un `MapaRealidad`; conceptualmente los utiliza para ejecutar la simulación.

• **Interfaz («cumple un contrato»):** distintas clases pueden implementar el mismo conjunto de operaciones sin tener que heredar de una misma clase base.

La herencia es apropiada cuando hay una relación de especialización real. La composición suele ser mejor para combinar responsabilidades independientes. Las interfaces son útiles para desacoplar código de implementaciones

concretas.

IV. Colecciones y LINQ

1. `List<T>` en el proyecto

`Cadete` declara `public List<string> Inventario { get; private set; }`. Es una lista genérica de cadenas que almacena nombres de objetos. El constructor la inicializa con `Célula de energía` y `Escudo`. Durante la exploración y el combate, el sistema puede añadir objetos mediante `Add()`; cuando se usa o elimina un objeto, utiliza `RemoveAt()`.

El método `MostrarInventario()` recorre la lista con un `for` y muestra cada elemento con su número. Esto es recorrido tradicional y sí está implementado.

2. Expresiones lambda y LINQ — teoría complementaria

En la versión revisada no se identifican consultas LINQ con `Where`, `Select` u `OrderBy`. Los siguientes ejemplos son para estudiar LINQ; no describen funcionalidad ya integrada.

Una expresión lambda es una función breve, por ejemplo `x => x.Length > 5`. LINQ permite expresar filtros, transformaciones y ordenamientos de colecciones.

```
List<string> inventario = new List<string>
{
    "Célula de energía", "Escudo", "Módulo de ataque"
};

var objetosLargos = inventario
    .Where(objeto => objeto.Length > 6)
    .OrderBy(objeto => objeto)
    .Select(objeto => objeto.ToUpper())
    .ToList();

foreach (string objeto in objetosLargos)
    Console.WriteLine(objeto);
```

Qué hace cada operación:

- `Where`: filtra los elementos que cumplen una condición.
- `OrderBy`: ordena ascendentemente por una clave.
- `Select`: transforma cada elemento en otro valor.
- `ToList`: materializa el resultado en una nueva lista.

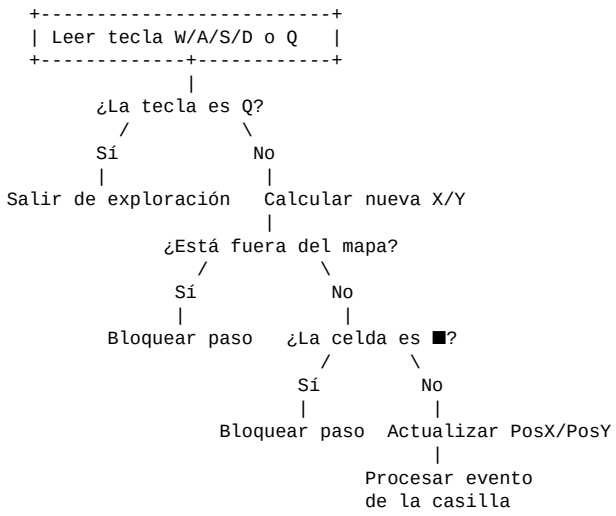
Para compilar el ejemplo se requiere `using System.Linq;` y el tipo `List<T>` necesita `using System.Collections.Generic;` si no está disponible mediante usings globales.

3. LINQ frente al recorrido tradicional

En NEXUS, `MostrarInventario()` usa un `for`, por lo que controla explícitamente el índice y el orden de impresión. LINQ puede hacer más declarativo un filtro u ordenamiento, pero no siempre sustituye un bucle: cuando se necesita modificar el estado paso a paso o manejar efectos laterales, un recorrido tradicional puede ser más claro.

V. Análisis del funcionamiento

1. Flujograma real: movimiento del cadete



Este flujo corresponde a `SistemaNexus.MenuMovimiento()` y al procesamiento posterior de eventos de casilla. Antes de mover al cadete, el programa valida que la posición nueva esté dentro de las dimensiones de `MapaRealidad` y que no contenga el símbolo ■. Si pasa ambas validaciones, actualiza `PosX` y `PosY` y llama al procesamiento del evento.

2. Seguimiento del estado de un objeto

Ejemplo basado en el constructor real de `Cadete`:

```
var cadete = new Cadete("Astra", 20, 70, 80);
```

Estado inicial esperado:

Propiedad	Valor	Razón
`Nombre`	`Astra`	Se envía como argumento al constructor base.
`Salud`	100	El constructor de `Cadete` llama a `base(nombre, 100)`.
`SaludMaxima`	100	El constructor de `Personaje` asigna la salud recibida como salud máxima.
`Edad`	20	Está dentro del rango de `Math.Clamp(edad, 16, 100)`.
`Energia`	70	Está dentro del rango de 0 a 100.
`Estabilidad`	80	Está dentro del rango de 0 a 100.
`Experiencia`	0	Se inicializa explícitamente a cero.
`Inventario`	Célula de energía, Escudo	Es la lista inicial creada en el constructor.

Si el sistema añade `"Módulo de ataque"` mediante `Inventario.Add(...)`, la lista pasa a tener tres elementos. Si después se ejecuta `Inventario.RemoveAt(1)`, se elimina el elemento del índice 1 (el segundo elemento), en este caso `Escudo`, siempre que la lista siga en ese orden.

3. Predicción del resultado de un fragmento de código

```
var cadete = new Cadete("Astra", 15, 120, 10);
Console.WriteLine(cadete.Edad);
Console.WriteLine(cadete.Energia);
Console.WriteLine(cadete.Estabilidad);
Console.WriteLine(cadete.Salud);
```

Resultado esperado:

```
16
100
```

```
10
100
```

Razonamiento: `Edad` se limita al mínimo de 16; `Energia` se limita al máximo de 100; `Estabilidad` permanece en 10 porque ya está entre 0 y 100; y `Salud` se inicializa en 100 a través del constructor base. En el flujo real de `SistemaNexus`, sin embargo, la validación de acceso rechaza la sesión porque la energía debe ser al menos 50 y la estabilidad al menos 20.

4. ¿Qué pasa si modificamos una condición?

En `SistemaNexus.Ejecutar()` se rechaza el acceso cuando `_cadete.Energia < 50 || _cadete.Estabilidad < 20`. Si se cambia el operador `||` (OR) por `&&` (AND), la condición solo rechazaría al cadete cuando ambas condiciones fueran verdaderas al mismo tiempo. Por ejemplo, con energía 40 y estabilidad 80, la condición original rechaza el acceso; con `&&`, no lo rechazaría por esa condición. Esto cambia la regla de acceso y permite iniciar el resto del flujo con energía menor al mínimo previsto.

Modificar una firma de método también puede provocar errores de compilación si los lugares que llaman al método no se actualizan. Modificar `EjecutarAtaqueEnemigo()` requiere respetar su tipo de retorno `int` y actualizar correctamente las implementaciones sobrescritas.

VI. Preparación para el examen: preguntas con respuestas razonadas

1. Comprensión de POO

Pregunta: ¿Por qué NEXUS utiliza clases en lugar de guardar todos los datos en variables locales dentro de `Main`?

Respuesta razonada: Las clases agrupan estado y comportamiento relacionado. `Cadete` mantiene salud, energía, posición e inventario; `MapaRealidad` gestiona la matriz; y `SistemaNexus` coordina la interacción. Esta separación facilita comprender y mantener el programa. Aunque el proyecto siga utilizando condiciones y bucles, las entidades se representan como objetos.

2. Análisis de clases y objetos

Pregunta: Si se crean dos objetos `Cadete` con nombres y energías diferentes, ¿comparten automáticamente sus valores de energía e inventario?

Respuesta razonada: No. Cada instancia tiene sus propios valores de instancia. Cada constructor crea una nueva lista `Inventario`, por lo que añadir un objeto a la lista de un cadete no añade automáticamente ese objeto a la lista del otro. Compartirían datos solo si el diseño utilizara explícitamente estado estático o referencias compartidas.

3. Herencia y polimorfismo

Pregunta: ¿Por qué `IniciarFaseCombatePokemon(Personaje enemigo)` puede recibir tanto un `Enemigo` como un `centinela`?

Respuesta razonada: Ambas clases heredan de `Personaje`, por lo que una referencia de tipo base puede referirse a cualquiera de los objetos derivados. La llamada a `EjecutarAtaqueEnemigo()` resuelve la implementación sobrescrita del tipo real del objeto, permitiendo distintos ataques sin escribir un método de combate completamente separado para cada enemigo.

4. Interfaces y diseño

Pregunta: Si NEXUS tuviera varios tipos de objetos recuperables, ¿qué ventaja podría ofrecer una interfaz `IRecuperable` frente a crear una jerarquía de herencia solo para recuperación?

Respuesta razonada: La interfaz permitiría que clases no relacionadas compartieran el contrato `Recuperar(int cantidad)`. El código cliente podría trabajar con ese contrato sin depender de una clase concreta. En la versión revisada de NEXUS, esta interfaz sería una mejora teórica: no está implementada en `Program.cs`.

5. Listas y LINQ

Pregunta: ¿Qué resultado produce la consulta `inventario.Where(x => x.Contains("Escudo")).OrderBy(x => x)` y en qué se diferencia del recorrido actual de `MostrarInventario()`?

Respuesta razonada: La consulta devuelve los elementos cuyo texto contiene `Escudo`, ordenados alfabéticamente. `Where` filtra y `OrderBy` ordena. El método actual `MostrarInventario()` no filtra ni ordena: recorre la lista existente por índice e imprime cada elemento en el orden en que está almacenado. La consulta LINQ es un ejemplo teórico complementario, no una consulta integrada en la versión revisada.

6. Predicción y modificación de código

Pregunta: ¿Qué cambia si la condición `Energia < 50 || Estabilidad < 20` se reemplaza por `Energia < 50 && Estabilidad < 20`?

Respuesta razonada: Con `||`, basta con que una variable incumpla el mínimo para rechazar el acceso. Con `&&`, se requiere que ambas incumplan simultáneamente. Por eso, un cadete con energía 40 y estabilidad 80 sería rechazado con la condición original, pero no por esa condición modificada. El cambio debilita la validación prevista.

7. Encapsulamiento y propiedades

Pregunta: ¿Qué efecto tiene `public int Edad { get; private set; }`?

Respuesta razonada: El código externo puede leer `Edad`, pero no asignarle un valor directamente a través del setter. La clase mantiene el control de la escritura y el constructor inicializa la edad mediante `Math.Clamp`. Esto ayuda a preservar la regla de que la edad quede entre 16 y 100.

8. Mapa y validación de movimiento

Pregunta: ¿Por qué `MenuMovimiento()` comprueba límites y obstáculos antes de actualizar `PosX` y `PosY`?

Respuesta razonada: La comprobación de límites evita intentar acceder a coordenadas fuera de la matriz. La comprobación del símbolo ■ evita atravesar una celda marcada como obstáculo. Solo si ambas validaciones pasan, el programa actualiza la posición y procesa el evento de la casilla. Si se quitara la validación, podrían producirse accesos inválidos o movimientos no permitidos.

Lista de verificación antes de entregar

☐ El archivo se guarda en `docs/NEXUS-Manual-Tecnico-H3.md`.

☐ El PDF se guarda en `docs/NEXUS-Manual-Tecnico-H3.pdf` si la entrega solicita PDF.

☐ Se identifica `Program.cs` como archivo principal revisado.

☐ Las relaciones del diagrama coinciden con las declaraciones reales de herencia.

☐ LINQ e interfaces están etiquetados como teoría complementaria, no como funcionalidades integradas.

☐ Las preguntas incluyen respuestas razonadas y cubren los temas exigidos.

☐ Si el código cambia, se actualizan los ejemplos y resultados del manual.

Fin del documento.